

Trabajo Práctico 1

Regresión e Introducción a la evaluación de modelos

72.75 — Aprendizaje Automático (Machine Learning)
Instituto Tecnológico de Buenos Aires

2026 Q2 · Defensa: 26/08/2026

Resumen

Se implementan regresión lineal y polinómica para predecir el costo anual de gastos médicos (**charges**) del dataset *Insurance Charges*, con validación cruzada de $k = 5$ folds. La partición train/test, el k -fold, la codificación *one-hot*, el estandarizado, la resolución por mínimos cuadrados, la expansión polinómica y el Lasso por descenso por coordenadas están implementados sobre `numpy`, sin `scikit-learn`, como pide el punto 2.2 del enunciado.

El modelo de producción es una **regresión lineal de grado 1 sin regularizar** con once características. Da un **RMSE de test de 4.288,52 dólares** ($R^2 = 0,871$). Es a la vez el de menor error de validación cruzada entre las 15 configuraciones elegibles y el más simple del espacio de búsqueda, así que no hay que elegir entre las dos cosas. La regla de 1 error estándar llega al mismo modelo: es el más simple de las 7 configuraciones indistinguibles del mínimo.

Tres de las once características —`fumador_obeso`, `edad_al_cuadrado` y `bmi_si_fuma`— salen del análisis exploratorio del punto 1 y representan la curvatura y las interacciones no aditivas del dataset. Con ellas incluidas, ni la expansión polinómica ni la regularización bajan el error. Se desarrolla en la sección 2.5.

Índice

1. Introducción teórica: separación train / validación / test	2
2. Punto 1 — Limpieza de datos	3
2.1. Distribución de las variables	3
2.2. Variables categóricas	3
2.3. Datos faltantes	4
2.4. Outliers	4
2.5. Las tres poblaciones de charges	5
2.6. Características y escalado	7
3. Puntos 2 y 3 — Implementación	8
3.1. El pipeline	8
3.2. Separación train / test	9
3.3. La regresión polinómica sigue siendo lineal en los parámetros	9
3.4. Regularización L_1	9
4. Punto 4 — Resultados	10
4.1. Sobreajuste por grado	11
5. Punto 5 — Respuestas	13
5.1. ¿Qué modelo obtuvo menor error?	13
5.2. ¿Cuál se implementaría en una aplicación real?	13
5.3. ¿Qué RMSE se espera en datos nuevos?	14
5.4. Los once coeficientes del modelo de producción	15
5.5. Sensibilidad al número de folds	16

5.6. Diagnóstico de residuos	18
6. Limitaciones	19
7. Guion de la presentación (10 minutos)	21
A. Reproducibilidad	21

1. Introducción teórica: separación train / validación / test

Un modelo de regresión se evalúa por lo que predice sobre datos que no vio. El error medido sobre el conjunto de entrenamiento no sirve para eso, porque los parámetros se ajustaron justamente para minimizarlo: un RMSE de train bajo puede significar que el modelo capturó el patrón o que memorizó el ruido de esas 1070 filas. Sin un conjunto separado no se distingue una cosa de la otra.

El trabajo separa tres roles, y ninguno reemplaza a otro:

- **Entrenamiento (train).** Ajusta los coeficientes de la regresión.
- **Validación.** No participa del ajuste. Se usa para elegir entre modelos: qué grado polinómico, qué valor de λ en Lasso.
- **Test.** Se evalúa una sola vez, al final, con grado y λ ya fijados. Estima el error sobre datos nuevos.

El test tiene que quedar sin usar hasta el final. En cuanto su error se usa para decidir algo —elegir un hiperparámetro, comparar dos configuraciones, volver a probar después de mirarlo— pasa a funcionar como un segundo conjunto de validación, y su RMSE deja de medir generalización: mide cuánto se ajustó el modelo también al test. En este trabajo el test (267 filas) se evaluó siempre después de fijar el modelo por validación cruzada. Ninguna decisión ajustable —grado, λ , características, umbral— se tomó mirando el test. La partición se reutilizó a lo largo del desarrollo; ese costo se detalla en *Limitaciones*.

Dónde se parte el dataset. La separación ocurre antes del análisis exploratorio, no justo antes de entrenar. El análisis exploratorio también decide: de él salen las tres características derivadas de la sección 2.5. Si esas decisiones se toman sobre el dataset completo, el test participó de ellas. El orden es:

1. limpieza de integridad —nulos y duplicados exactos— sobre el dataset completo;
2. separación train/test;
3. análisis exploratorio y elección de características, **sólo sobre train**.

El primer paso va antes del split porque no toma ninguna decisión estadística: dos filas idénticas en las 7 columnas son un defecto de carga, y dejarlas repartidas entre train y test rompe la independencia del test. Todo lo demás —distribuciones, outliers, correlaciones, el umbral de `fumador_obeso`— se mide sobre las 1070 filas de entrenamiento.

Por qué validación cruzada y no un único split. Con un solo split el resultado depende de qué filas cayeron del lado de validación. Con 1070 filas de train, una partición particular puede favorecer o perjudicar a un modelo según cómo se repartieron los outliers. La validación cruzada promedia sobre varias particiones: se parte el train en 5 bloques de unas 214 filas y se repite el ciclo ajustar–validar 5 veces, dejando cada vez un bloque distinto fuera del ajuste. Cada dato valida una vez y entrena en las otras cuatro vueltas. El promedio de los 5 errores tiene menos varianza que el error de un único split, y esa reducción es lo que permite distinguir

si una configuración generaliza mejor que otra. Con un n chico como este, sin esa reducción las diferencias entre configuraciones quedan por debajo del ruido.

2. Punto 1 — Limpieza de datos

2.1. Distribución de las variables

El primer paso es mirar estadísticas descriptivas e histogramas juntos: las descriptivas dan los valores, los histogramas la forma de cada distribución. La figura 1 cubre las siete variables.

Todo lo de esta sección está medido sobre las 1070 filas de train, por el motivo de la sección 1. La única excepción es el control de integridad de la sección 2.3, que va antes del split.

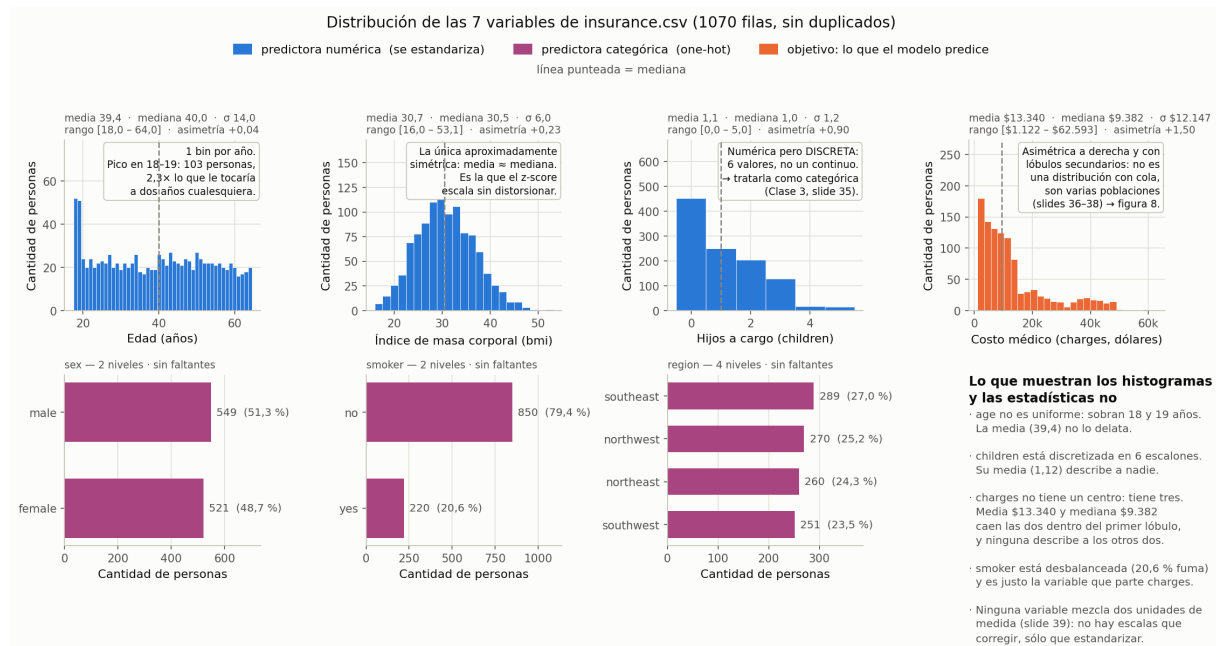


Figura 1: Las siete variables. Arriba las cuatro numéricas, con histograma; abajo las tres categóricas, con frecuencias. El color codifica el rol de cada variable en el modelo. Las numéricas enteras (**age**, **children**) van con un bin por valor; las continuas (**bmi**, **charges**), con el ancho de la regla de Freedman–Diaconis, que se apoya en el rango intercuartil y no se infla con la cola de **charges**.

Tres observaciones que las medias no muestran:

- **age no es uniforme.** Hay 103 personas de 18 y 19 años, unas 2,3 veces lo que le correspondería a dos años cualesquiera. La media (39,4) no lo indica.
- **children es discreta:** seis escalones enteros. Se puede tratar como categórica o como numérica; se evaluaron las dos formas y se conserva numérica (sección 2.6).
- **charges tiene tres modos.** Asimetría +1,50 y lóbulos secundarios claros. Es el hallazgo que define el modelo; se desarrolla en la sección siguiente.

Ninguna variable muestra dos escalas superpuestas, así que no hay unidades de medida mezcladas que corregir. Sí hay escalas distintas, que se resuelven con estandarizado más abajo.

2.2. Variables categóricas

El dataset tiene tres variables categóricas: **sex** (binaria), **smoker** (binaria) y **region** (cuatro categorías: *northeast*, *northwest*, *southeast*, *southwest*). Se codificaron con **one-hot** y no con codificación ordinal. La codificación ordinal impone un orden y una distancia numérica que no

existen en los datos: asignar 0, 1, 2, 3 a las regiones le dice al modelo que *southwest* está más lejos de *northeast* que de *southeast*, cuando la variable es nominal.

Para **region** se generaron 4 dummies y se **descartó una**, que queda como categoría de referencia absorbida en la ordenada al origen. Es la forma estándar de evitar la *dummy variable trap*: con las 4 dummies presentes, su suma es siempre 1, con lo cual son linealmente dependientes de la columna constante, la matriz de diseño queda singular y $X^T X$ no se puede invertir.

Para las binarias alcanza con **una columna** (**sex=male**, **smoker=yes**). La segunda categoría es el complemento exacto de la primera, así que agregarla produce la misma colinealidad con dos categorías en vez de cuatro.

Se descartaron dos alternativas al one-hot. Codificar cada categoría por su frecuencia no aporta nada con cuatro categorías de tamaños parecidos. Codificarlas por el promedio de **charges** usa la variable objetivo para construir la entrada, con lo cual habría que recalcularla dentro de cada fold para no filtrar información de validación; con cuatro regiones, ese costo no compra nada frente a tres columnas binarias.

El codificador produce 11 columnas: **age**, **bmi**, **children**, **fumador_obeso**, **edad_al_cuadrado**, **bmi_si_fuma**, **sex=male**, **smoker=yes**, **region=northwest**, **region=southeast**, **region=southwest**.

2.3. Datos faltantes

Este es el único análisis del punto 1 que corre sobre el dataset completo, porque detectar nulos y duplicados no requiere ninguna decisión estadística.

No hay valores nulos en las 1338×7 celdas originales, así que no hace falta imputar ni eliminar filas por faltantes. Sí hay **una fila duplicada exacta**: los índices 195 y 581 son idénticos en las 7 columnas. Se elimina uno de los dos y quedan 1337 filas, 1070 de train y 267 de test.

La eliminación va **antes del split**. Un duplicado exacto repartido entre train y test filtra información del test al entrenamiento: el modelo no tiene que generalizar nada para esa fila y su error de test queda artificialmente bajo.

2.4. Outliers

Se comparó la detección por **rango intercuartílico (IQR)** contra **z-score**. Se optó por IQR. Los dos criterios discrepan:

Tabla 1: Outliers detectados por cada criterio, sobre train.

Variable	IQR detecta	Z-score detecta
charges	115 (10,7 %)	5
children	0	16

En **charges** el z-score detecta menos porque depende de la media y el desvío estándar, y **charges** tiene asimetría 1,50. Los valores extremos arrastran la media hacia arriba e **inflan el desvío**, con lo cual el umbral de tres desvíos se corre justo por efecto de los valores que se quiere detectar. El IQR usa cuartiles y no tiene esa dependencia.

En **children** pasa lo inverso: el z-score marca 16 valores y el IQR ninguno. **children** es un entero acotado entre 0 y 5, y estar a tres desvíos de la media no significa lo mismo en una variable discreta de rango chico que en una continua sin cota.

Los outliers se conservan. De los 115 outliers de **charges** detectados por IQR, el **97,4 % son fumadores**, contra 11,3 % de fumadores en el resto de la población. Esa desproporción indica que **no son errores de carga sino una subpoblación real**: fumar aumenta el costo médico de forma no lineal, y esos 115 casos son la cola derecha de esa subpoblación. Eliminarlos

sacaría del entrenamiento el efecto más grande del dataset, que es además el que más importa predecir bien.

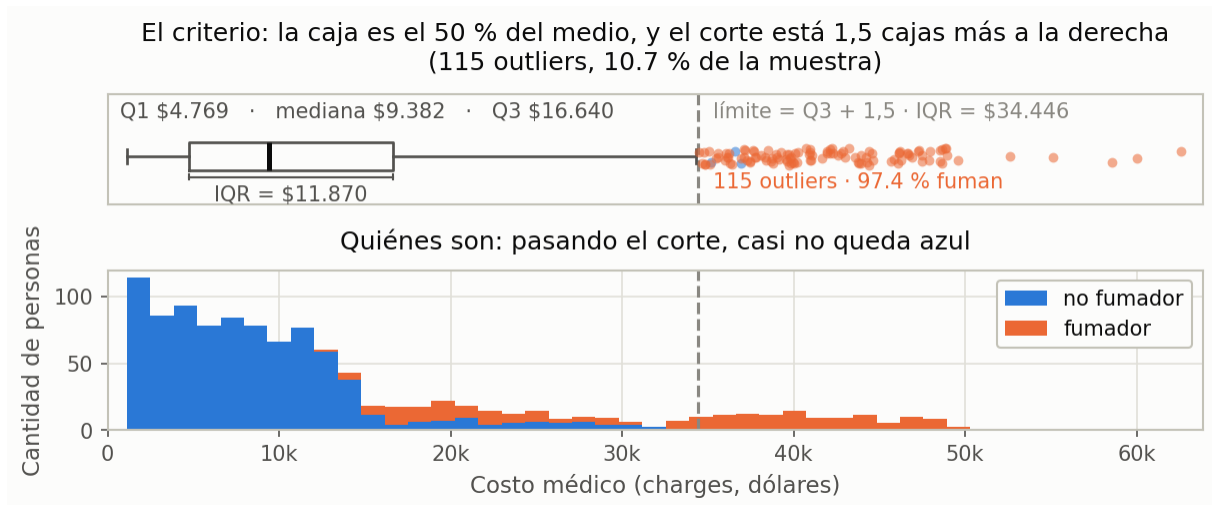


Figura 2: Arriba: el criterio. La caja va de Q_1 a Q_3 —el 50 % central— y su ancho es el IQR; el corte está una caja y media más a la derecha. Abajo: la misma distribución apilada por condición de fumador. Los dos paneles comparten el eje horizontal, así que el umbral punteado es una sola vertical. La cola por encima del corte es 97,4 % fumadores, que es el argumento para conservarla.

2.5. Las tres poblaciones de charges

El histograma de **charges** muestra un pico grande y dos jorobas a la derecha. Con más de una población hay dos caminos: (a) dar esa información al modelo como variable binaria, o (b) partir el dataset y entrenar un modelo por población. Antes de elegir hay que verificar que las poblaciones existen, mirando el histograma separado por grupo. La figura 3 es ese paso.

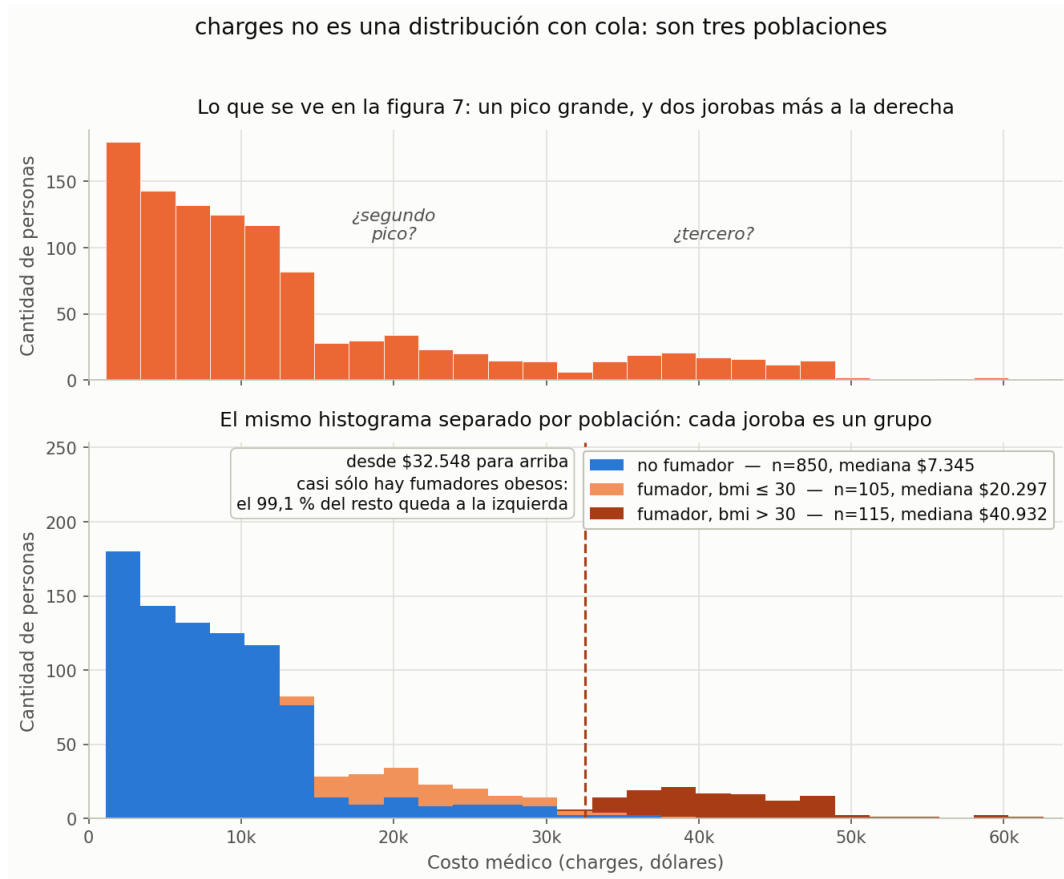


Figura 3: Arriba, `charges` sin separar. Abajo, el mismo histograma con los mismos bins, partido por población. Las medianas son \$7.345, \$20.297 y \$40.932.

Son tres y no se separan igual de bien. El grupo de fumadores con $\text{bmi} \leq 30$ (105 personas) se solapa con la cola de los no fumadores (850). El de fumadores con $\text{bmi} > 30$ (115) arranca en \$32.548, por encima del 99,1 % de todos los demás. Se eligió el camino (a): con 105 y 115 filas, entrenar un modelo por población deja a dos de los tres con muy pocos datos, mientras que una columna binaria da la misma información a un único modelo.

Es un escalón, no una pendiente. Partiendo el bmi en tramos finos alrededor de 30, entre los fumadores el costo medio salta de \$24.079 en el tramo $[29, 30)$ a \$38.878 en el $[30, 31)$: casi quince mil dólares en una unidad de bmi , con pendiente suave a los dos lados del corte. Entre los no fumadores el mismo corte casi no mueve el costo (\$8.563 contra \$8.089). Si fuera una pendiente, los saltos entre tramos contiguos serían parecidos.

Eso importa porque la expansión polinómica de grado 2 ya genera el término $\text{bmi} \times \text{smoker} = \text{yes}$, y un producto con una variable continua sólo representa un cambio de pendiente, no un salto. Por eso se agrega la característica binaria

$$\text{fumador_obeso} = (\text{smoker} = \text{yes}) \wedge (\text{bmi} > 30),$$

El umbral 30 es el de la OMS: es una constante médica externa al dataset, y esa procedencia es lo que hace legítimo usarla, porque no sale de buscar qué corte minimiza el error. El barrido de umbrales sobre train coincide: el RMSE de validación cae a \$4.455 en $\text{bmi} > 30$ y sube a los dos lados (\$4.776 en 29, \$4.888 en 31).

No hay fuga de datos: las dos columnas de las que sale están disponibles al momento de predecir y `charges` no interviene en el cálculo. El efecto viene de la *interacción*, no de sus partes:

agregar sólo $\text{bmi} > 30$ deja el error donde estaba (\$6.039 contra \$6.094 de base), mientras que la interacción completa lo baja a \$4.455.

2.6. Características y escalado

El enunciado pide dos decisiones justificadas. Van por separado.

Qué características se incluyen: las seis originales, sin descartar ninguna, más tres derivadas. El enunciado admite que “pueden ser todas”, y en este dataset esa es la respuesta correcta para las seis variables crudas. Se suman `edad_al_cuadrado` y `bmi_si_fuma`, que la tabla documenta junto con las demás porque forman parte de la misma decisión (`fumador_obeso` ya se justificó en la sección 2.5). Las tres derivadas se midieron sobre train, con validación cruzada de 5 folds promediada sobre 8 particiones. Partiendo de las 9 características con `fumador_obeso`: `edad_al_cuadrado` baja el RMSE de validación de \$4.455 a \$4.425, `bmi_si_fuma` a \$4.410, y las dos juntas a \$4.380.

Tabla 2: Decisión de inclusión, variable por variable.

Variable	¿Se incluye?	Por qué
<code>age</code>	Sí	Segunda correlación más alta con <code>charges</code> (0,309) y relación monótona clara. En el modelo de producción queda con coeficiente 45,10.
<code>bmi</code>	Sí	Correlación baja (0,194) pero no descartable: su efecto depende de <code>smoker</code> , y Pearson mide relación lineal, no interacciones. Coeficiente 111,20.
<code>children</code>	Sí	Correlación 0,101, la más débil. Se conserva numérica y no como categórica: con <i>one-hot</i> el RMSE de validación empeora en todos los casos medidos (de \$4.380 a \$4.385 en OLS grado 1, y de \$4.406 a \$4.497 en el mejor Lasso). Coeficiente 815,02.
<code>sex</code>	Sí	Binaria, una columna. El modelo de producción no regulariza, así que no hay penalización que la apague: queda con coeficiente -218,32 (<code>sex=male</code>), de los más chicos en magnitud entre los once, pero no nulo.
<code>smoker</code>	Sí	La variable más informativa del dataset (correlación 0,786).
<code>region</code>	Sí	Tres columnas tras el one-hot. Aporta poco, pero es la única variable geográfica y su costo es mínimo.
<code>edad_al_cuadrado</code>	Sí	Derivada: representa la <i>curvatura</i> del costo con la edad, que un término lineal en <code>age</code> no puede representar. Coeficiente 3749,13, el tercero más grande de los once.
<code>bmi_si_fuma</code>	Sí	Derivada: representa la <i>pendiente</i> del <code>bmi</code> , distinta entre fumadores y no fumadores. Coeficiente 5236,99, el más grande de los once.

Costo de interpretabilidad. `edad_al_cuadrado` es una función determinista de `age`, y `bmi_si_fuma` lo es de `bmi` dentro del grupo de fumadores. Están muy correlacionadas con sus variables de origen. Por eso los coeficientes de `age` (45,10), `bmi` (111,20) y `smoker=yes` (1161,22) quedan chicos: las derivadas absorbieron parte del efecto. Eso no significa que `age` o `bmi` dejen de importar, sino que dentro de cada grupo colineal (`age` con `edad_al_cuadrado`; `bmi`, `smoker=yes`)

y `fumador_obeso` con `bmi_si_fuma`) los coeficientes individuales no se interpretan por separado: hay que leer el grupo completo. Es un costo real a cambio de mejor ajuste.

El criterio general es que **no se descarta ninguna variable a mano**. Con 1070 filas de train y 11 columnas tras codificar no hay problema de dimensionalidad que lo justifique. Descartar por correlación baja sólo mira relaciones lineales de a una variable por vez, y este dataset tiene el caso contrario: `bmi` correlaciona 0,194 con `charges` y en interacción con `smoker` es la fuente del efecto más grande del modelo. Cuando hace falta seleccionar —en los grados altos, donde la expansión genera cientos de columnas— la selección se delega a la penalización L_1 , que la hace **dentro de cada fold** y por lo tanto sin fuga.

El escalado. Las variables numéricas se **estandarizan** (z -score: media 0, desvío 1) antes de ajustar cualquier modelo con regularización, porque Lasso penaliza la magnitud de los coeficientes. Con las variables en escalas distintas —`age` en años, `bmi` en kg/m^2 , `children` en unidades enteras chicas— la penalización castiga más a las variables que necesitan coeficientes grandes por su unidad, no por su importancia.

Se eligió z -score y no min-max al rango $[0, 1]$ porque el trabajo **conserva** los 115 outliers de `charges`. Min-max fija los extremos de la escala en el mínimo y el máximo observados, así que un solo valor extremo comprime a todos los demás contra un costado del rango. El z -score usa media y desvío, que se mueven mucho menos con un puñado de valores altos.

Para el modelo de producción —lineal, sin regularizar— estandarizar no cambia las predicciones: es una transformación afín de las columnas y mínimos cuadrados la absorbe en los coeficientes. Sí cambia cómo se leen esos coeficientes, y por eso la tabla 7 es interpretable: con todas las características en desvíos, las magnitudes son comparables.

Media y desvío se calculan **sobre el fold de entrenamiento únicamente**, nunca incluyendo el fold de validación. Es la misma lógica que con el test: si se calcularan sobre todos los datos, el escalado ya habría visto los datos de validación antes de que el modelo intentara predecirlos.

3. Puntos 2 y 3 — Implementación

3.1. El pipeline

1. `quitar_duplicados`: $1338 \rightarrow 1337$ filas.
2. `separar_train_test` con semilla 42: 1070 filas de train, 267 de test (80/20). **Acá se congela el test**: el análisis exploratorio que eligió las características del paso siguiente corrió sólo sobre estas 1070 filas (`src.datos.cargar_train`).
3. `agregar_derivadas`: agrega `fumador_obeso`, `edad_al_cuadrado` y `bmi_si_fuma`. El código las aplica al DataFrame entero antes de indexar train y test, y eso da lo mismo: son funciones fijas fila a fila de `age`, `bmi` y `smoker`, no aprenden ningún parámetro de la muestra y ninguna usa `charges`, así que aplicarlas antes o después del split da las mismas columnas. Lo que sí se decidió sobre train es *cuáles* tres columnas agregar y con qué umbral.
4. `CodificadorCategoricas`, ajustado **sólo con train**: produce las 11 columnas.
5. `k_fold`: partición propia en 5 folds sobre el train.
6. Dentro de cada fold: `Estandarizador` (ajustado sólo con el fold de entrenamiento) $\rightarrow$ `expandir_polinomial` $\rightarrow$ `Estandarizador` otra vez $\rightarrow$ ajuste del modelo.
7. Modelos: `RegresionLineal` (mínimos cuadrados vía descomposición en valores singulares) y `Lasso` (descenso por coordenadas con actualización incremental del residuo).

3.2. Separación train / test

Se genera una **permutación aleatoria** de los índices $0 \dots 1336$ y se corta: las últimas $\text{round}(1337 \times 0,2) = 267$ posiciones van a test, las 1070 restantes a train. Tres decisiones dentro de eso:

- **Se baraja antes de cortar.** Tomar las últimas 267 filas del archivo sería válido sólo si el orden fuera aleatorio, y los CSV suelen venir ordenados por algún criterio. Barajar rompe cualquier estructura en el orden.
- **La semilla es fija (42).** Sin semilla fija ningún número de este informe sería reproducible. También evita correr varias veces y quedarse con la partición que da mejores números, que es una forma encubierta de ajustar contra el test.
- **No se estratifica, y el motivo se midió.** El argumento inmediato sería que **charges** es continuo y no hay clases que preservar, pero el análisis exploratorio lo contradice: **charges** son *tres poblaciones casi disjuntas* (figura 3), y los cinco folds reparten la tercera de forma desigual —entre 17 y 31 personas, del 7,9 % al 14,5 %—, con correlación +0,62 entre esa proporción y el **charges** medio del fold. Hay clases de facto y se reparten mal. Sin embargo, estratificar por ellas no cambia el resultado, y eso se midió sobre train: promediando sobre ocho particiones, el desvío del RMSE entre folds es 593 con folds aleatorios y 591 con folds estratificados. La diferencia es de dos dólares, cuando entre dos particiones del mismo método va de 410 a 715. Lo que mueve el error de un fold no es cuántos fumadores obesos le tocaron sino cuáles: dentro de ese grupo los costos van de \$32.548 a \$63.770, así que igualar los conteos deja intacta la varianza que importa.
- **No hay estructura temporal ni de grupos.** Cada fila es una persona distinta, así que el supuesto de independencia que justifica el barajado se cumple.

3.3. La regresión polinómica sigue siendo lineal en los parámetros

Elevar **age** al cuadrado o multiplicar **bmi** por **smoker=yes** no cambia la naturaleza del modelo: sigue siendo una combinación lineal de columnas de la matriz de diseño, sólo que esas columnas son transformaciones no lineales de las variables originales. Lo que se ajusta por mínimos cuadrados es el mismo problema que con grado 1. Lo único que cambia es qué características recibe el modelo.

3.4. Regularización L_1

El objetivo que minimiza el Lasso implementado es

$$J(w) = \frac{1}{2n} \|y - Xw - b\|_2^2 + \lambda \|w\|_1, \quad (1)$$

resuelto por descenso por coordenadas con *soft-thresholding*. Se usa L_1 y no L_2 (Ridge) porque acá la penalización tiene que hacer dos trabajos: controlar la magnitud de los coeficientes y elegir características. En grado 4 la expansión genera 1364 columnas; L_2 las dejaría a todas vivas, apenas encogidas. L_1 lleva la mayoría exactamente a cero —17 sobrevivientes en promedio con $\lambda = 310,83$ — y devuelve un modelo legible. El intercepto b no se penaliza: se obtiene centrando X e y y recuperándolo como $b = \bar{y} - \bar{x}^T w$. Penalizarlo encogería la predicción media hacia cero, que no es el objetivo de la regularización.

La grilla de λ se construye **relativa a $\lambda_{\text{máx}}$** , el menor valor que anula todos los coeficientes:

$$\lambda_{\text{máx}} = \max_j \frac{|x_j^T (y - \bar{y})|}{n}. \quad (2)$$

Se calcula sobre el train completo y da 10360,94, el mismo valor para los grados 2, 3 y 4. Una grilla relativa es comparable entre grados; una absoluta elegida a ojo, no.

4. Punto 4 — Resultados

Tabla 3: Validación cruzada de 5 folds sobre las 1070 filas de entrenamiento. Regresión lineal sin regularizar, por grado del polinomio. Tabla generada por `src/tablas.py` a partir de `resultados/cv_lineal.csv`.

Grado	RMSE train	RMSE validación	Características
1	4351,6 $\pm$ 93,2	4413,4 $\pm$ 367,0	11
2	4227,2 $\pm$ 104,7	4516,7 $\pm$ 380,0	77
3	3931,1 $\pm$ 117,1	6757,5 $\pm$ 857,0	363
4	3379,1 $\pm$ 105,3	87916,9 $\pm$ 47000,1	1364

Tabla 4: Lasso, mismo esquema de validación cruzada. Las configuraciones que no convergieron quedan excluidas de la selección. Tabla generada por `src/tablas.py` a partir de `resultados/cv_lasso.csv`.

Grado	λ	RMSE train	RMSE validación	Coefs. $\neq$ 0
2	3108,28	6369,5	6397,0	4,4
2	1036,09	4711,7	4730,3	5,8
2	310,83	4408,7	4434,0	9,4
2	103,61	4330,4	4422,9	22,2
2	31,08	4288,7	4459,2	37,4
3	3108,28	6363,7	6390,7	4,2
3	1036,09	4711,0	4730,1	5,4
3	310,83	4389,3	4431,4	15,6
3	103,61	4260,7	4464,4	48,8
3	31,08	4152,0	4621,8	96,0
4	3108,28	6363,4	6394,4	4,2
4	1036,09	4709,9	4732,4	5,6
4	310,83	4378,6	4433,7	17,0
4	103,61	4183,2	4531,6	74,2
4	31,08	3942,5	4938,2	166,6

4.1. Sobreajuste por grado

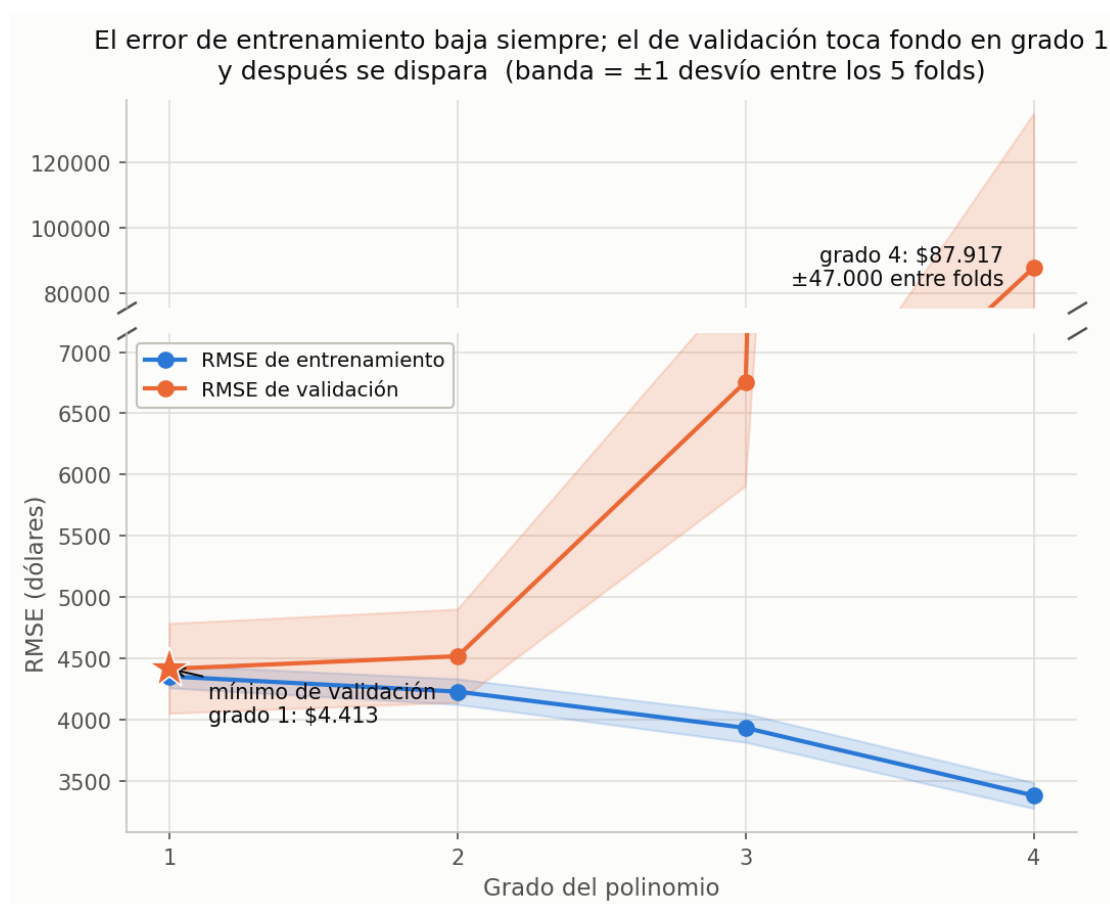


Figura 4: El error de entrenamiento baja monótonamente con el grado; el de validación toca fondo en grado 1 y después sube. El eje está *partido*: el grado 4 llega a 87.916,9 dólares y en una escala única aplastaría a los otros tres. La banda es ± 1 desvío entre folds.

En regresión lineal sin regularizar el sobreajuste empieza en grado 2. El error de **train** baja monótonamente (4351,6 $\rightarrow$ 4227,2 $\rightarrow$ 3931,1 $\rightarrow$ 3379,1 dólares), porque más características dan más grados de libertad para ajustar lo ya visto. El de **validación** hace lo contrario desde el arranque: mínimo en grado 1 (4413,4) y sube en cada grado siguiente, hasta 87916,9 en grado 4, **19,9 veces** el del grado 1.

Por qué el grado 4 falla. Se puede anticipar contando parámetros contra datos. Una regla práctica pide del orden de diez ejemplos por parámetro, y cada fold entrena con 856 filas.

Tabla 5: Cuántos datos pediría cada grado, contra los 856 que hay por fold de entrenamiento.

Grado	Parámetros	Ejemplos que pediría (10 $\times$)	¿Alcanzan las 856 filas?
1	11	110	sí
2	77	770	justo
3	363	3.630	no
4	1364	13.640	no, y además $p > n$

La tabla predice el orden de la columna de validación de la tabla 3. El grado 4 es el caso extremo: con más columnas que filas el problema **queda indeterminado**. El diagnóstico de

rango de `src/experimentos.py` lo confirma: de las 1364 columnas, el rango efectivo es 436, o sea que el **68,0 %** son combinación lineal exacta de las otras (las causas están en *Limitaciones*; una es que a partir de grado 2 la expansión polinómica vuelve a generar `edad_al_cuadrado` y `bmi_si_fuma`, que ya están entre las once). En grado 1 no hay duplicación: cada característica aparece una vez y las once columnas son de rango completo. Con 856 filas por fold, el ajuste sobre el espacio de grado 4 es una extrapolación.

El indicador más claro de sobreajuste no es sólo que el RMSE de validación suba, sino que la **brecha** entre train y validación crezca: de **61,8 dólares** en grado 1 a **84.537,9 dólares** en grado 4. Un modelo que generaliza bien tiene error de validación parecido al de train; una brecha que se ensancha indica que el modelo está aprendiendo particularidades del fold de entrenamiento.

El segundo indicador es la **estabilidad entre folds**: el desvío del RMSE de validación pasa de $\pm 367,0$ en grado 1 a $\pm 47,000,1$ en grado 4, **128 veces más**. Eso dice algo distinto de que el modelo sea peor en promedio: dice que el grado 4 es **inestable**, que su desempeño cambia según qué filas caen en cada partición.

El desvío en grado 1. Los $\pm 367,0$ del modelo elegido no son chicos. Al representar el escalón de `fumador_obeso`, el modelo elimina el error sistemático que dominaría sin esa columna, y lo que queda es el error de los casos extremos, cuya repartición entre folds varía de una partición a otra. Es el mismo fenómeno que explica por qué estratificar no ayuda.

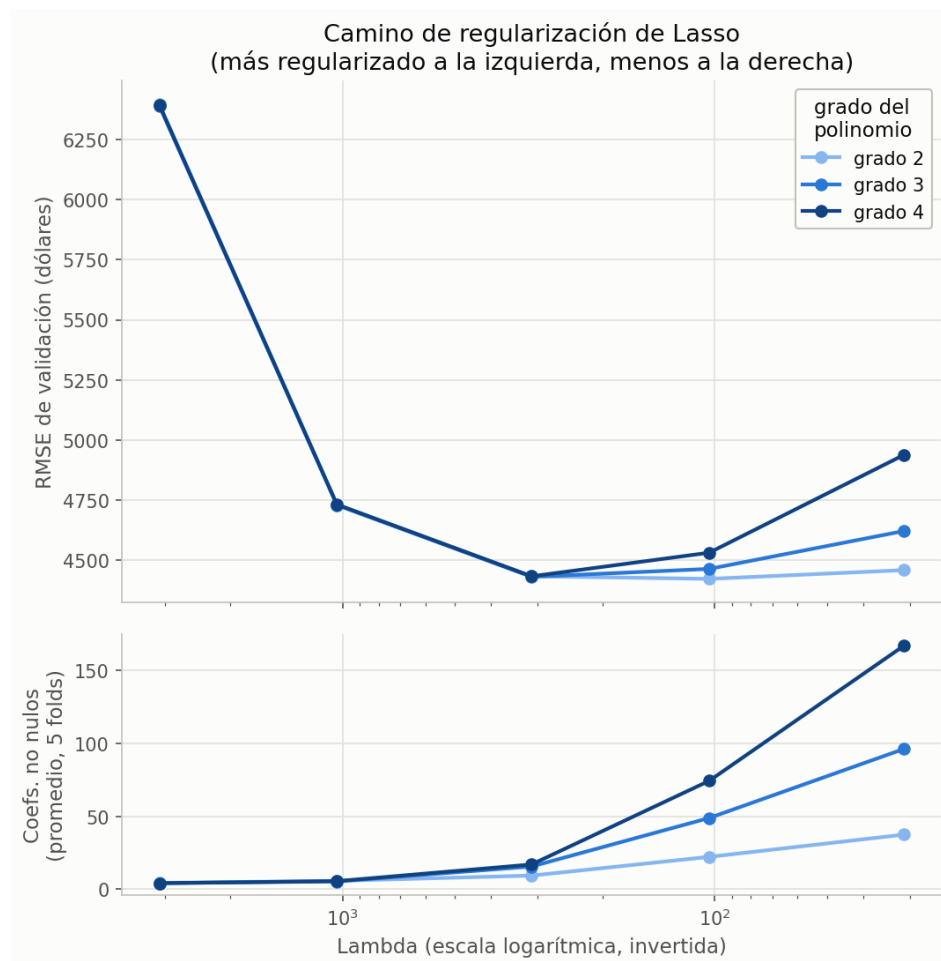


Figura 5: Camino de regularización. Al aflojar λ entran más características (panel inferior) y el error primero baja y después vuelve a subir. El tono codifica el grado del polinomio: más oscuro, grado más alto.

Lasso corrige buena parte del problema. El mejor λ de grado 4 que converge ($\lambda = 310,83$)

lleva el RMSE de validación de 87916,9 a 4433,7 dólares, con 17,0 de los 1364 coeficientes vivos en promedio. Recuperar de un espacio degenerado un error comparable al del modelo lineal simple es lo que la regularización L_1 aporta acá.

Lo que no logra es superarlo. El mejor Lasso de toda la tabla (grado 2, $\lambda = 103,61$) queda en 4422,92, nueve dólares por encima del lineal de grado 1, con 22,2 características vivas en promedio en lugar de 11. Sobre esa comparación decide el punto siguiente.

5. Punto 5 — Respuestas

5.1. ¿Qué modelo obtuvo menor error?

El de menor RMSE de validación cruzada es la **regresión lineal de grado 1 sin regularizar**: RMSE de validación = $4413,45 \pm 366,97$, con las once características de grado 1, todas vivas. El mejor Lasso de la búsqueda (grado 2, $\lambda = 103,61$) queda nueve dólares por encima, en 4422,92, con 22,2 coeficientes vivos en promedio.

5.2. ¿Cuál se implementaría en una aplicación real?

El mismo modelo. No hay que elegir entre el de menor error y el más simple porque son la misma configuración.

Igual se corre el procedimiento completo. Aplicando la **regla de 1 error estándar** sobre el ganador,

$$ES = \frac{366,97}{\sqrt{5}} = 164,11, \quad \text{umbral} = 4413,45 + 164,11 = 4577,56, \quad (3)$$

De las 19 configuraciones corridas, **cuatro** no convergieron y quedan fuera de la comparación: Lasso de grado 3 con $\lambda = 310,83$, $\lambda = 103,61$ y $\lambda = 31,08$, y Lasso de grado 4 con $\lambda = 31,08$. De las 15 elegibles, **7 caen por debajo del umbral**. Esas 7 son, para los datos disponibles, **indistinguibles entre sí**: cuál aparece primera en la tabla depende de cómo cayó la partición en 5 folds, no de una diferencia real de capacidad predictiva.

Ante un empate estadístico corresponde quedarse con la configuración **más simple** del grupo. Ese paso acá no cambia nada: la más simple de las 7 —once características, grado 1, sin regularizar, sin hiperparámetros que ajustar, y la de menos columnas en la matriz de diseño— ya es la de menor error de validación cruzada de toda la búsqueda. La regla de 1 ES confirma la elección del mínimo en lugar de modificarla. Como el ganador de la CV y el modelo de producción son el mismo, el costo de la simplicidad es **0,00 dólares**.

El resultado central es este: *con las características correctas, ni la expansión polinómica ni la regularización bajan el error*. Las dos existen para dar flexibilidad de representar relaciones que las variables crudas no representan. Una vez que `fumador_obeso`, `edad_al_cuadrado` y `bmi_si_fuma` las representan de forma explícita, la flexibilidad extra sólo agrega varianza. La evidencia está en el punto 1: sin esas columnas derivadas el modelo lineal de grado 1 se queda corto —\$6.094 de RMSE de validación contra los \$4.380 que alcanza con ellas— y hace falta subir a grado 2 con Lasso para acercarse.

Tabla 6: Resultado sobre el conjunto de test: las 267 filas que no participaron de ninguna decisión. Se reportan las dos métricas: el RMSE en dólares, que es el que pide el enunciado, y R^2 , que lo compara contra la variabilidad propia de `charges`. El baseline predice siempre la media de train.

Modelo	Caract.	RMSE test	R^2 test
Producción (lineal grado 1, sin regularizar)	11	4.288,52	0,871
Baseline: predecir siempre la media	—	11.963,43	0,000

El modelo reduce el error del baseline a algo más de un tercio y explica el **87,1 %** de la varianza de **charges** en datos que no vio. Esa mejora no viene de agregar grados de libertad al polinomio ni de ajustar un hiperparámetro de regularización —las dos cosas empeoraron el error de validación— sino de las tres características derivadas del análisis exploratorio del punto 1.

5.3. ¿Qué RMSE se espera en datos nuevos?

4.288,52 dólares — el RMSE de **test** del modelo de producción,
no su RMSE de validación (4.413,45).

El número de validación está sesgado a la baja y no se puede prometer. Esa configuración se eligió por tener el RMSE de validación más bajo de las 15 estimaciones elegibles (de 19 corridas; cuatro quedaron fuera por no converger). Cada estimación tiene ruido de muestreo, porque es un promedio sobre 5 folds. Al elegir el **mínimo** de un conjunto de estimaciones ruidosas, ese mínimo queda sesgado hacia abajo aunque cada estimación individual sea insesgada: es más probable que el ganador lo sea en parte por una racha favorable de ruido. Acá el argumento aplica sin salvedades, porque no hay un modelo elegido por simplicidad y otro que ganó la CV: hay uno solo, y es el mínimo de la tabla. El test, al no haber participado de ninguna decisión, no tiene ese sesgo.

El test dio mejor que la validación: 4.288,52 contra 4.413,45, una diferencia de $-124,93$ dólares. Eso no contradice lo anterior: el sesgo de selección predice que el test tienda a ser peor *en promedio*, no en una partición puntual, y 124,93 dólares es chico frente al $\pm 366,97$ de ruido que ya separaba a los folds entre sí. Con una sola partición de test no se puede determinar si el ruido jugó a favor esta vez o si el sesgo es más chico de lo que la teoría sugiere.

Dos advertencias sobre ese número.

- El conjunto de test tiene **267 filas**. El 4.288,52 es una estimación con su propia incertidumbre de muestreo, que este trabajo no cuantificó con un intervalo de confianza. No corresponde leerlo con cuatro decimales.
- El RMSE **promedia sobre una población heterogénea**. El error no se reparte parejo: se concentra en los fumadores, que son los casos más caros. Un único RMSE global no muestra esa asimetría.

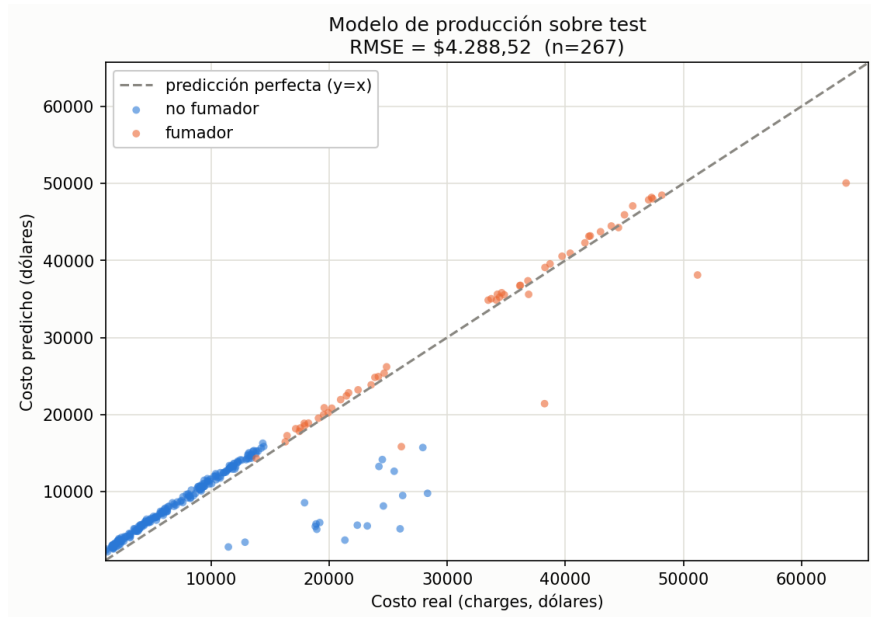


Figura 6: Predicho contra real sobre el conjunto de test. El modelo acierta en la mayoría barata (nube azul sobre la diagonal) y se dispersa en los casos caros, que son casi todos fumadores: es la asimetría del error que menciona la segunda advertencia.

5.4. Los once coeficientes del modelo de producción

Tabla 7: El modelo completo. Once características de grado 1, sin penalización: no hay selección que reportar. Los coeficientes están sobre características estandarizadas, así que sus magnitudes son comparables entre sí, con una salvedad: `age` y `edad_al_cuadrado` son función determinista una de la otra, y lo mismo `bmi`, `smoker=yes` y `bmi_si_fuma`. Dentro de cada grupo colineal el coeficiente individual no se interpreta solo (ver *Costo de interpretabilidad*).

Característica	Coficiente
<code>bmi_si_fuma</code>	5236,99
<code>fumador_obeso</code>	4891,83
<code>edad_al_cuadrado</code>	3749,13
<code>smoker=yes</code>	1161,22
<code>children</code>	815,02
<code>region=southwest</code>	-498,36
<code>region=southeast</code>	-424,86
<code>sex=male</code>	-218,32
<code>region=northwest</code>	-142,53
<code>bmi</code>	111,20
<code>age</code>	45,10

Los tres coeficientes más grandes son las tres características derivadas: `bmi_si_fuma` (5236,99), `fumador_obeso` (4891,83) y `edad_al_cuadrado` (3749,13), las tres por encima de `smoker=yes` (1161,22), que sobre las variables crudas es el término dominante. La lectura, en orden: la pendiente del `bmi` entre fumadores es lo que más pesa; ser fumador y obeso suma otro tanto; la curvatura de la edad importa casi tanto como esas dos; y recién después aparece el efecto de fumar por sí solo, reducido porque parte de lo que explicaba antes —la pendiente distinta según el `bmi`— ahora lo toma `bmi_si_fuma`. Es el mismo fenómeno del costo de interpretabilidad: las características derivadas absorben el efecto de las originales.

El contraste está en lo que hace el Lasso cuando no recibe `fumador_obeso` y tiene que arreglarse con la expansión polinómica: deja vivo el término `bmi*smoker=yes` entre los de mayor magnitud, por encima de `bmi` sola. Encuentra la interacción por su cuenta, que es para lo que sirve L_1 , pero la encuentra en la única forma que su espacio de características permite, como un cambio de pendiente. El análisis exploratorio mostró que la forma correcta era un escalón, y darle esa columna directamente al modelo dio mejor resultado que la maquinaria que la aproximaba.

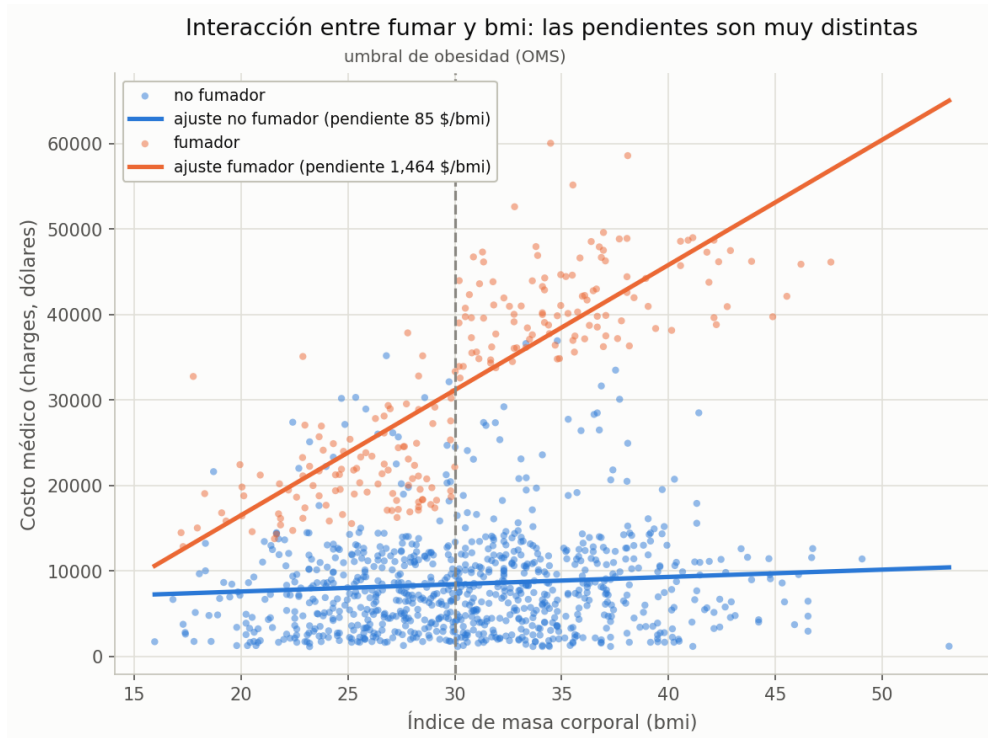


Figura 7: La interacción, cuantificada. Ajustando una recta por grupo, la pendiente es de **83 dólares por unidad de BMI** entre los no fumadores y de **1473** entre los fumadores: un factor de **17,7**. Un modelo aditivo sobre las variables crudas no puede representar esa diferencia; `fumador_obeso` sí, con un solo coeficiente.

5.5. Sensibilidad al número de folds

Toda la sección anterior descansa sobre $k = 5$. Corresponde medir cuánto de la respuesta depende de esa elección.

Qué se mide y qué no. Repetir la *selección completa* —las 19 configuraciones para varios valores de k — cuesta horas por cada k , dominadas por el Lasso de grado 4 sobre 1364 columnas. Este trabajo no reporta esa corrida. Lo que sí se mide es el **barrido controlado**: se fija la configuración de producción y se varía sólo k , con lo cual la única causa de las diferencias es k . Los resultados están en `resultados/sensibilidad_k.csv` y en la figura 8.

Nada de esto toca el test. El módulo `src/sensibilidad_k.py` rehace el mismo split con la misma semilla y descarta `idx_test` sin usarlo: todo ocurre dentro de las 1070 filas de train.

Sensibilidad al número de folds — configuración de producción fija (lineal grado 1, sin regularización)

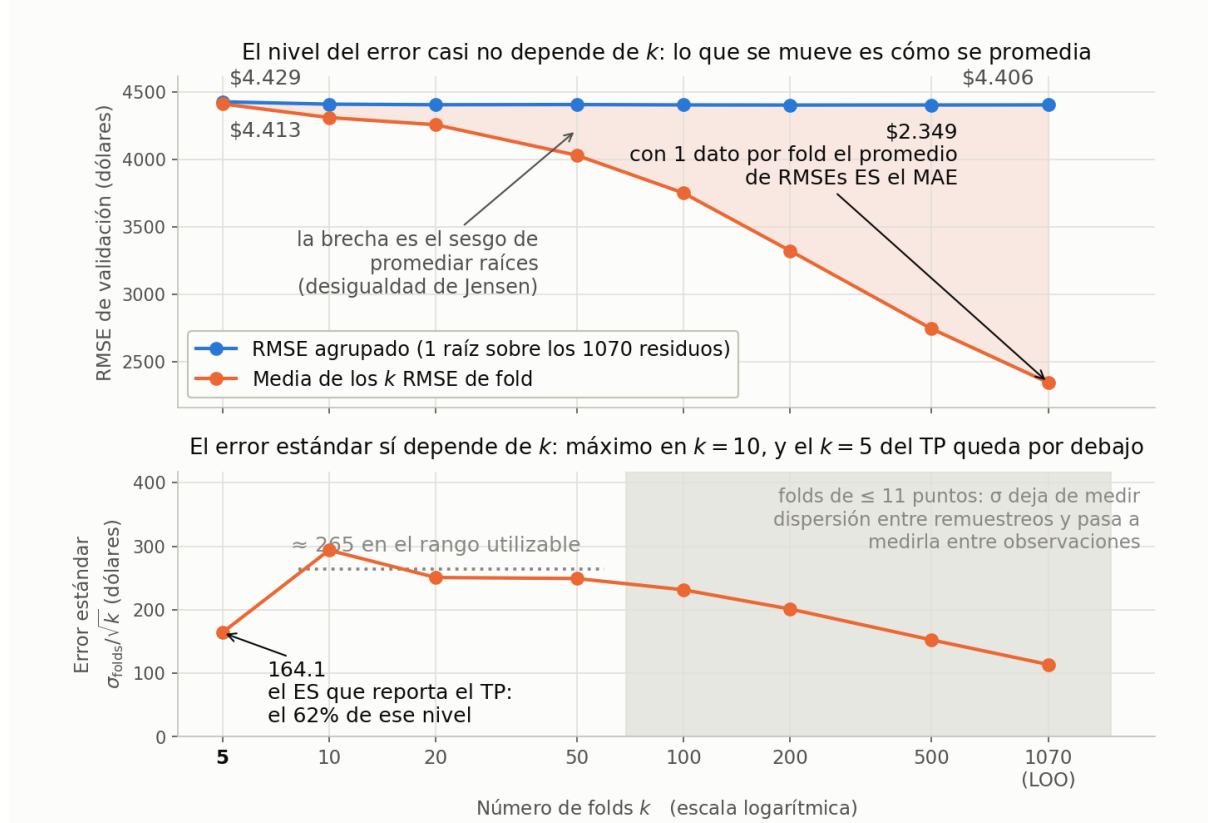


Figura 8: Sensibilidad al número de folds, con la configuración de producción fija (lineal grado 1, sin regularizar, once características). Los ocho valores de k dejan folds de 214, 107, 54, 21, 11, 5, 2 y 1 punto. **Arriba:** el nivel del error casi no depende de k —la curva agrupada va de 4428,7 a 4405,9, un 0,51 %—; lo que cambia es la forma de promediarlo, y el área sombreada entre las dos curvas es ese sesgo. **Abajo:** el error estándar sí depende de k , con un máximo de 293,9 en $k = 10$ y una caída posterior; el $k = 5$ del informe vale 164,1. La banda gris marca dónde los folds son tan chicos que σ deja de medir dispersión entre remuestreos.

Primero: la caída del RMSE es un artefacto de la métrica, no del modelo. El pipeline reporta la *media de los k RMSE de fold*, y esa media baja 2064,9 dólares (4413,4 $\rightarrow$ 2348,5) al ir de $k = 5$ a LOO. No es que $k = 5$ sea fuertemente pesimista: la raíz es cóncava, así que por desigualdad de Jensen

$$\text{media}_i \sqrt{\text{ECM}_i} < \sqrt{\text{media}_i \text{ECM}_i}, \quad (4)$$

con una brecha que crece a medida que los folds se achican. Calculando el **RMSE agrupado** —una sola raíz sobre los 1070 residuos *out-of-fold* juntos, que no tiene ese sesgo— el error va de 4428,7 a 4405,9: **22,8 dólares, un 0,51 %**, y agotados ya en $k = 20$. El sesgo pesimista real de $k = 5$ es despreciable.

El mismo fenómeno en el extremo es lo que vuelve inservible a *leave-one-out* con este pipeline: con un dato por fold, $\text{RMSE}_i = |y_i - \hat{y}_i|$ y el promedio de los k RMSE es el **MAE** (2348,5), un 46,7 % por debajo del RMSE agrupado del mismo k (4405,9) y no comparable contra el RMSE de test de este informe. LOO no es más validación: cambia la métrica sin avisar, además de exigir ajustar cada configuración 1070 veces en lugar de 5.

Segundo: el error estándar tiene un único pico, en $k = 10$. Sobre la grilla densa — $k = 5, 10, 20, 50, 100, 200, 500, 1070$ — el ES sube de 164,1 en $k = 5$ a un máximo de 293,9 en $k = 10$, y de ahí cae de forma monótona: 250,9 / 249,3 / 231,6 / 201,3 / 152,8 / 114,0 hasta

LOO. No hay meseta entre $k = 10$ y $k = 50$: el pico está en un solo punto y la curva ya baja en $k = 20$.

El mecanismo se verifica contra los datos. Si σ (el desvío del RMSE entre folds) creciera exactamente como $\sqrt{k}$, el ES sería constante. De $k = 5$ a 10, σ salta $2,53\times$ ($367,0 \rightarrow 929,5$) donde $\sqrt{k}$ predice $1,41\times$ —la σ de $k = 5$ está estimada con *cinco* números y `ddof=0`, un estimador ruidoso y sesgado hacia abajo—, y ese exceso empuja el ES hacia arriba. Pasado el pico la relación se invierte: σ sigue creciendo pero en promedio más lento que $\sqrt{k}$ (de $k = 500$ a LOO salta $1,09\times$ donde $\sqrt{k}$ predice $1,46\times$), porque con folds de 2 o 1 punto σ deja de medir variabilidad entre remuestreos y satura contra la dispersión de los residuos individuales, que es la banda gris de la figura 8.

Los valores son dato medido; que las causas sean esas dos es la lectura más razonable del patrón, no una descomposición formal de la varianza. Lo que queda establecido es que el pico, 293,9 en $k = 10$, es $1,79$ veces el 164,1 que la regla de 1 ES usó en la selección del punto 5.

Consecuencia. No se cambia el esquema. El nivel del error no depende de k una vez corregida la forma de promediar, así que el RMSE de validación del punto 4 no es un artefacto de haber puesto 5. El error estándar sí depende de k : con $k = 10$ habría sido $1,79$ veces el 164,1 usado en el punto 5. Eso no debilita la elección del modelo simple: con un error estándar más ancho hay más configuraciones estadísticamente indistinguibles del mejor, y la más simple sigue siendo la misma.

5.6. Diagnóstico de residuos

El modelo de producción llega a 4.288,52 dólares de RMSE de test. La pregunta siguiente es si ese error se puede bajar y con qué. La contesta un diagnóstico *out-of-fold* sobre train: para cada una de las 1070 filas, la predicción sale de un modelo que no la vio durante el ajuste —el mismo esquema de 5 folds del punto 4, pero guardando el residuo de cada fila en vez de sólo el RMSE agregado— en `src/diagnostico_residuos.py` (`resultados/diagnostico_residuos.csv`). No construye `X_test`: es un diagnóstico sobre train, no una evaluación más del modelo.

Tabla 8: Concentración del error cuadrático (SSE) sobre las filas de train, ordenadas de mayor a menor residuo.

Fracción de filas	Filas	% del SSE total
1 %	10	23,53 %
5 %	53	75,38 %
10 %	107	91,54 %

El error no está repartido. Diez personas —el 1 % del train— explican casi una cuarta parte del error cuadrático total. El 5 % explica tres cuartas partes. El modelo no falla parejo: acierta en la mayoría de las filas y falla mucho en un núcleo chico.

Tabla 9: Reparto del error cuadrático por población. El error se concentra en los **no fumadores**, no en los fumadores obesos que motivaron `fumador_obeso`.

Población	n	RMSE	% del SSE total
No fumadores	850	4615	86,3 %
Fumadores, $\text{bmi} \leq 30$	105	3407	5,8 %
Fumadores, $\text{bmi} > 30$	115	3804	7,9 %

Por población. Dentro de “no fumadores” hay otra subpoblación de facto que ni `fumador_obeso` ni `bmi_si_fuma` distinguen, porque las dos características derivadas están construidas alrededor de `smoker=yes`.

El núcleo: 28 personas. Filtrando a los no fumadores con `charges > 25,000` —28 filas, 2,6 % del train— ese grupo aporta el **41,9 % del error cuadrático total**, y el modelo los **subestima en promedio 17.378 dólares**. Es el bloque de error más grande del dataset.

No se los puede identificar con las columnas disponibles:

Tabla 10: Perfil de las 28 personas del núcleo contra el resto de los no fumadores. La única diferencia apreciable es la edad.

Variable	Estas 28 personas	Resto de los no fumadores
age (media)	50,79	39,11
bmi (media)	31,23	30,69
children (media)	1,39	1,10
sex	54 % F / 46 % M	51 % F / 49 % M
region	pareja a la del resto	pareja a la del resto

Bmi, children, sexo y región dan el mismo perfil que el resto de los no fumadores. Sólo la edad se separa (50,8 contra 39,1), y no alcanza para aislar al grupo. Ninguna combinación de las columnas originales —ni sumando las tres derivadas— lo separa del resto antes de ver `charges`.

Es error irreducible con este dataset. Ese ~ 42 % del error cuadrático **no es falta de capacidad del modelo**: falta la variable que explica el gasto —una patología, una cirugía, una condición preexistente— y no está en el CSV. Ninguna familia de modelos puede recuperar una señal que no está en las columnas de entrada; agregar grados de libertad ahí sólo memoriza ruido, que es el sobreajuste que documenta el punto 4 en los grados 3 y 4.

El diagnóstico corre sobre el pipeline de once características, con las tres derivadas incorporadas, y aun así ese núcleo concentra el 41,9 % del error, porque las tres derivadas están construidas alrededor de `smoker=yes` y no tocan a este grupo. **Para bajar ese tramo del error hace falta otra variable, no otro modelo.**

6. Limitaciones

- **Colinealidad en grados altos.** En grado 4, el 68,0 % de las 1364 columnas son redundantes (rango efectivo 436). Hay tres causas. Primera: una dummy binaria elevada a una potencia sigue teniendo dos valores posibles, así que es una función afín exacta de la dummy original —por ejemplo $\text{smoker=yes}^2 = 1,456866 \cdot \text{smoker=yes} + 1,0$, con un residuo de $1,5 \times 10^{-14}$, exacto salvo error de punto flotante—. Segunda: el producto de dos dummies del mismo one-hot cae dentro del espacio generado por las dummies individuales más la constante, porque 3 dummies y una constante ya generan todas las funciones posibles

sobre las 4 regiones. Tercera: la expansión polinómica **duplica** dos de las características derivadas a partir de grado 2 — age^2 coincide con `edad_al_cuadrado`, y $\text{bmi} \cdot \text{smoker} = \text{yes}$ con `bmi_si_fuma`—, lo que lleva la redundancia al 68,0 % y explica que cuatro configuraciones Lasso no converjan. En grado 1 no hay duplicación: las once columnas son de rango completo.

Ese producto **no es idénticamente cero**: lo sería en la codificación cruda 0/1, pero el pipeline estandariza antes de expandir, así que cada dummy toma valores distintos de 0 y 1 y el producto también. Es redundante por la segunda razón, no por ser nulo. La consecuencia práctica es que **dentro de un grupo colineal el reparto de coeficientes es arbitrario**: no se interpretan de a uno cuando el grado es alto.

- **Una sola partición train/test, con una sola semilla.** El RMSE de test reportado tiene varianza asociada a qué filas cayeron del lado de test, que este trabajo no midió. Repetir el split con otra semilla daría un número distinto, presumiblemente cercano.
- **La partición de test se reutilizó durante el desarrollo.** A medida que el conjunto de características se fue cerrando, el test se evaluó más de una vez sobre las mismas 267 filas. En todas las corridas la elección del modelo se hizo con validación cruzada sobre train: ni `src/experimentos.py`, ni `src/evidencia_features.py`, ni `src/diagnostico_residuos.py` construyen `X_test`, y eso se verifica con un `grep`. Aun así el costo es real: el protocolo pide una partición nueva cada vez que cambia el modelo, y cada evaluación extra sobre la misma erosiona su independencia.
- **El umbral de fumador_obeso no se validó como hiperparámetro.** Se tomó el de la OMS, que es externo al dataset. El barrido de umbrales lo confirma *a posteriori*, pero si el corte se hubiera elegido por ese barrido sería un hiperparámetro ajustado contra validación y el RMSE de validación quedaría sesgado a la baja.
- **El dataset es simulado.** Proviene de estadísticas demográficas —el origen es el dataset del libro de Lantz— y no del registro de una aseguradora real, así que las relaciones que el modelo aprendió reflejan cómo se construyó la simulación.
- **El RMSE penaliza el error al cuadrado**, con lo cual la métrica está dominada por los casos caros. Un modelo puede tener un RMSE relativamente bueno y aun así errar de forma proporcionalmente grande en los casos baratos.

7. Guion de la presentación (10 minutos)

Tabla 11: Reparto del tiempo. La introducción teórica y el punto 5 son lo que más se evalúa.

Min.	Qué se dice	Figura
0-1	Dataset (<code>insurance.csv</code> , 1338 filas) y la pregunta del TP: predecir <code>charges</code> y elegir un modelo defendible, no sólo el de menor error.	—
1-3	Introducción teórica: por qué <code>train</code> no alcanza, qué rol cumple cada conjunto, por qué el <code>test</code> se toca una sola vez, y qué gana la validación cruzada frente a un único <code>split</code> .	—
3-4	Punto 1: dónde se parte el dataset y por qué el EDA va después; por qué se conservan los 115 outliers (97,4 % fumadores) y por qué IQR y no <code>z-score</code> .	04-outliers
4-5	Puntos 2 y 3: el pipeline y por qué la regresión polinómica sigue siendo lineal en los parámetros.	—
5-7	Punto 4: las curvas por grado, dónde aparece el sobreajuste (la brecha de 61,8 a 84.537,9, el desvío de ± 367 a ± 47.000) y cómo Lasso lo corrige.	01-curvas, 02-camino
7-9	Punto 5: por qué el lineal grado 1 gana la CV siendo además el más simple del espacio, por qué 7 de las 15 son indistinguibles y confirman la misma elección, y el RMSE prometido ($R^2 = 0,871$), con el sesgo del mínimo.	una-es
9-10	Limitaciones y cierre.	—

A. Reproducibilidad

El repositorio corre con Python 3.10 o superior y solamente `numpy`, `pandas` y `matplotlib`. No usa `scikit-learn` ni `pytest`.

Comando	Qué hace
<code>python -m src.datos</code>	Punto 1: análisis exploratorio
<code>python -m src.experimentos</code>	Puntos 2 a 5 (unos 27 minutos)
<code>python -m src.evidencia_features</code>	La evidencia numérica de las tres derivadas, de la estratificación y de <code>children</code>
<code>python -m src.diagnostico_residuos</code>	El diagnóstico de residuos y el error irreducible
<code>python -m src.graficos</code>	Genera las figuras de este informe
<code>python -m src.tablas</code>	Regenera las tablas del punto 4 desde los CSV
<code>python -m src.sensibilidad_k</code>	Barrido de k de la sección 5.5
<code>python -m tests.test_validacion</code>	Suite del módulo de partición y métricas
<code>python -m tests.test_preproceso</code>	Suite de codificación, escalado y expansión
<code>python -m tests.test_modelos</code>	Suite de los modelos
<code>python -m tests.test_paleta</code>	La paleta de las figuras, bajo simulación de daltonismo

Los resultados están versionados en `resultados/` (`cv_lineal.csv`, `cv_lasso.csv`, `modelo_elegido.json`, `evaluacion_test.json`, `evidencia_features.csv`, `diagnostico_residuos.csv` y `sensibilidad_k.csv`), junto con la salida completa de la corrida en `informe/salida-seleccion.txt`, de modo que no hace falta correr nada para verificar los números de este informe.

Procedencia del dataset. El archivo se descargó del enlace que da el enunciado ([kaggle.com/datasets/ mirichoi0218/insurance](https://kaggle.com/datasets/mirichoi0218/insurance)) y se verificó que la copia usada es byte a byte idéntica al original publicado:

SHA256 = 388eff679557d08ac19f463d025de5e0b4adc482537c8456d19934d78621fd47